



Project Title	Rising Waters: Machine Learning Based Flood Prediction System		
Developer	B. Meghana Tej	Team ID	[Team ID]
Date	28 to 29 June	Phase / Document	Project Development Phase

12. Code Layout, Readability & Reusability

Good code layout matters even in a solo project, since the developer is effectively their own future collaborator: any confusion introduced now (an unclear variable name, a hidden dependency between files) has to be re-understood by the same person days later while under time pressure. The structure and practices below were chosen with that in mind, favoring explicitness over cleverness.

12.1 Project Structure

Path	Purpose
app.py	Flask application: loads model/scaler, serves the UI, handles /predict requests
train.py	Standalone training pipeline: cleaning, scaling, model comparison, serialization
notebook.ipynb	Exploratory data analysis and experimentation
data/flood_prediction.csv	Historical rainfall/weather dataset with flood label
models/flood_model.joblib	Serialized best-performing trained classifier
models/scaler.joblib	Serialized StandardScaler fitted on training data
templates/index.html	Single-page frontend (form, results, Auto-Fill logic)
README.md	Setup, usage, and feature documentation

12.2 Readability & Reusability Practices

- Clear separation of concerns: train.py (offline training) is fully decoupled from app.py (online serving).
- Column renaming in train.py explicitly maps raw dataset headers (e.g. “Jun-Sep”, “ANNUAL”) to the exact field names used in the Flask form (jun_sep, annual_rainfall), keeping the two layers in sync.
- Descriptive naming throughout (input_dict, features_df, features_scaled, flood_prob) makes the prediction flow traceable without additional comments.
- Model artifacts are persisted with Joblib so training does not need to be repeated on every server start, supporting reuse across sessions/deployments.
- Error handling in the /predict route wraps input parsing and prediction in a try/except block, returning a readable error message instead of a server crash.
- train_and_evaluate() is encapsulated as a single callable function guarded by an if __name__ == "__main__" entry point, allowing it to be imported and reused elsewhere if needed.

Taken together, these choices mean a new reader can understand the system by reading two files — train.py for how the model comes to exist, and app.py for how it is used — without needing to trace execution across a deeper module hierarchy.

